

Teaching Automated Deduction: an Implementation-Based Approach with Maude

Romain Sidhoum and Simon Robillard

LIRMM, Université de Montpellier, CNRS, Montpellier, France

1 Introduction

Automated deduction tools have been used to teach mathematical reasoning [1, 3, 4, 6], either to give feedback or to guide students in their proof attempts. In this work, we consider a different but related question: how to teach automated deduction? This question was raised in the context of “Automated Software Verification”, a course offered at the University of Montpellier that covers various algorithms and methods used in automated verification, including sequents and tableaux-based methods for propositional and first-order logic, SAT solving, SMT solving, and saturation-based theorem proving. The target audience consists of first year computer science master’s students enrolled in either the *algorithms* program or the *software engineering* program. Until now, it was taught through lectures and exercise sessions during which the students applied the algorithms with pen-and-paper, but no programming exercises. Because of this, some students, particularly among those less inclined to theoretical computer science, found it difficult to engage with the course. We sought to remedy this with practical exercises aimed at implementing the techniques covered in the course. However, in the spirit of the course, these programming exercises should focus on the algorithms themselves, rather than attempting to program an efficient theorem prover. Implementation matters such as term indexing or optimized data structures, for example, are essential to the performance of real-world provers, but they add complexity to an already difficult topic, and are therefore out of the scope of the course.

We strove to find an implementation language that would offer a high-level abstraction and allow students to program in a style that would closely mirror the textbook presentation of the algorithms. We eventually chose Maude [2], a specification and programming language based on rewriting logic that features a powerful pattern-matching system. It offers several features that allow very high-level descriptions of systems, notably (1) pattern matching and rewriting modulo associativity and commutativity (AC), (2) subtyping and polymorphic functions, (3) separation between rewriting rules and their application strategies. Rewriting modulo AC allows the syntactic definitions of objects such as clauses, formulas or (multi)sets to carry some semantics, and greatly facilitates the handling of such objects. Subtyping is beneficial for code reuse, but can also be used in more subtle ways, e.g., to specify the stopping condition of a recursive algorithm. Finally, the definition of application strategies for rewriting rules allows separation of concerns, e.g., between the definition of an inference calculus and the proof search strategy that uses it.

We gave the twelve students of this course a series of exercises¹ that guide them step by step toward the implementation of a theorem prover for first-order logic, starting with formula pre-processing functions, and ending with the implementation of the resolution calculus and a simple saturation algorithm. The students were provided with the basic architecture of the prover, ensuring that they started with correct definitions and did not have to worry about

¹Available online at <https://gite.lirmm.fr/rsidhoum/maude-satu>

boilerplate code or architectural concerns (e.g., module definitions and imports). They worked on this material during two sessions spanning four and a half hours. We first describe the assignment (Section 2) then discuss the results observed in the classroom (Section 3).

2 The assignment

2.1 Introduction to Maude

To introduce students to the Maude programming language, we begin by providing a typed representation of first-order formulas using Maude’s syntax. This encoding makes extensive use of sub-sorts and operator overloading, which are central features of Maude’s type system. These features allow the abstract syntax of logical formulas to be encoded in a way that closely mirrors their hierarchical structure as described in textbooks. Below is the algebraic specification provided in the worksheet:

```

sorts Atom Literal Clause CNF Formula .
subsorts Atom < Literal < Clause < CNF < Formula .

op p : String TermList -> Atom .
op bot : -> Atom .
op top : -> Atom .
op ~_ : Atom -> Literal [prec 53] .
op ~_ : Formula -> Formula [prec 53] .
op _/\_ : CNF CNF -> CNF [assoc comm prec 55 id: top] .
op _/\_ : Formula Formula -> Formula [ditto] .
op _\/_ : Clause Clause -> Clause [assoc comm prec 59 id: bot] .
op _\/_ : Formula Formula -> Formula [ditto] .
op _=>_ : Formula Formula -> Formula [prec 61] .
op fa : String Formula -> Formula .
op ex : String Formula -> Formula .

```

In this specification, Atoms are the most basic syntactic elements and are modeled as the most specific sort. Literals, clauses, conjunctive normal forms (CNFs), and formulas are progressively more general sorts, with sub-sorting relationships encoding their syntactic hierarchy.

The use of polymorphic operators (e.g., $\wedge$ and $\vee$) across multiple sorts allows the same syntax to be used uniformly for both general and restricted forms of formulas. This supports a natural encoding of formulas and enables students to define functions that operate on logical constructs at multiple levels of abstraction without additional boilerplate. Additionally, conjunction and disjunction are defined as associative commutative operators with a neutral element (respectively `top` and `bot`).

This design helps bridge the gap between textbook logic and executable specifications. It leverages Maude’s support for pattern matching modulo associativity and commutativity (AC), which simplifies the manipulation of formulas by abstracting away syntactic grouping.

2.2 Exercises on the syntax

The worksheet begins with a series of exercises focused on the manipulation of formulas and terms. Each exercise targets a key component that will be part of the prover. Students are asked to implement the following functions in Maude:

1. `clausify : Formula  $\rightarrow$  CNF`. Transforms a first-order formula into CNF, using De Morgan's laws and distributivity.
2. `substitution : Formula  $\times$  Substitution  $\rightarrow$  Formula`. Applies a substitution, given as a set of variable-term pairs, to all relevant positions in a formula.
3. `unify : Formula  $\times$  Formula  $\rightarrow$  Substitution?`. Computes a most general unifier (MGU) for two formulas, returning either a unifier or a failure indicator.
4. `skolem : Formula  $\rightarrow$  Formula`. Computes a skolem normal form.

To illustrate how the capabilities of Maude allow the implementation of these functions in a way that is close to how they would be written in a logic textbook, below is a possible solution for the `clausify` function:

```
eq clausify(F:CNF) = F:CNF .
eq clausify(~ ~ F) = clausify(F) .
eq clausify(~ (F1 /\ F2)) =
  clausify(clausify(~ F1) \/ clausify(~ F2)) .
eq clausify(~ (F1 \/ F2)) =
  clausify(~ F1) /\ clausify(~ F2) .
eq clausify(F1 => F2) = clausify(clausify(~ F1) \/ clausify(F2)) .
eq clausify(F1 \/ (F2 /\ F3)) =
  clausify(F1 \/ F2) /\ clausify(F1 \/ F3) .
```

This implementation aims to recursively simplify a formula by applying rewrite rules until a CNF is obtained. Using sub-sorts declaration and polymorphism, no additional checks are needed to determine whether a formula is already in CNF: if the formula has sort CNF, the first equation applies.

Below is an implementation of the `unify` function, based on Robinson's unification algorithm.

```
op unify : UnificationProblem -> Substitution? .
eq unify(problem-nil) = identity .
eq unify(problem-unsolvable) = no-unifier .
eq unify(T =? T ; UP) = unify(UP) .
eq unify(f(F, TL1) =? f(F, TL2) ; UP) =
  unify(decompose(TL1, TL2) ; UP) .
ceq unify(f(F, TL1) =? f(G, TL2) ; UP) = None if F /= G .
eq unify(f(F, TL) =? v(X) ; UP) = unify(v(X) =? f(F, TL) ; UP) .
ceq unify(v(X) =? T ; UP) =
  unify(replace(v(X), T, UP))
  if not occurs(v(X), T) /\ occurs(v(X), UP) .
```

```

ceq unify(v(X) =? T ; UP) = None if occurs(v(X), T) .
eq unify(UP) = UP [owise] .

op unify : TermList TermList -> Substitution? .
eq unify(TL1, TL2) = unify(decompose(TL1, TL2)) .

```

This implementation models the unification process as a sequence of transformations on a unification problem. Using pattern matching modulo AC allows to non-deterministically select an equation and to apply the corresponding rule.

2.3 Exercises on the solver

A saturation theorem prover [5] works on a set of clauses and each step adds inferred new clauses until either the empty clause is derived, indicating a contradiction, or until the clause set is saturated, meaning no new clause can be added. To infer new clauses, the prover uses a chosen set of inference rules, such as ordered resolution or superposition.

The final set of exercises focuses on implementing the resolution calculus, as well as a saturation algorithm, yielding a refutation complete theorem proving procedure for first-order logic. The students are asked to implement two main functions:

1. inference-step : $\text{CNF} \times \text{Clause} \rightarrow \text{CNF}$.

This function computes all new clauses that can be inferred using a given clause as primary premise and a set of clauses for additional premises. These inferences rules applied are:

- Resolution:

$$\frac{L_1 \vee \Gamma_1 \quad \neg L_2 \vee \Gamma_2}{(\Gamma_1 \vee \Gamma_2)\phi}$$

where ϕ is a most general unifier (MGU) of literals L_1 and L_2

- Factoring:

$$\frac{L_1 \vee L_2 \vee \Gamma}{(L_1 \vee \Gamma)\phi}$$

where ϕ is an MGU of L_1 and L_2

The goal of this function is to model inference rules in a way that mirrors their textbook definitions by relying on Maude's support for matching modulo associativity and commutativity. This allows clauses to be treated as sets, enabling concise and declarative rule definitions.

These inference rules are refutation complete for first-order logic. Although this means that no other rules are needed, usually simplification rule are applied: removing redundant clauses and tautologies. Pruning the set of clauses is crucial in real life theorem provers as the set of clauses grow exponentially. However, our aim is to make students write a prover that is able to solve a smaller, curated, set of formulas. This allow students to spend more time on the principles of the algorithm. It also allows faster students that are able to finish the worksheet sooner to further improve the prover by incorporating these simplification rules as a bonus exercise.

2. saturation : $\text{CNF} \times \text{CNF} \rightarrow \text{Nat}$.

The saturation function encodes the main saturation loop. It takes two clause sets:

- a *new* clause set, from which inferences are still to be made.
- an *old* clause set, which has already been processed.

The algorithm proceeds iteratively:

- Select a clause from the *new* set.
- Compute all inferences between this clause and the clauses in the *old* set using inference-step.
- Add the inferred clauses to the *new* set.
- Move the selected clause to the old set.

The process continues until one of the following occurs:

- The empty clause is derived, indicating a contradiction. In that case, the function returns 1.
- The new set is empty and no new clauses can be generated. This means that the clause set is saturated but satisfiable. The function returns 0.

Below is a possible solution for the inference-step function:

```

op inference-step : CNF Clause -> CNF .
op infer-resolution-pos : CNF Clause -> Clause .
op infer-resolution-neg : CNF Clause -> Clause .
op infer-factoring : Clause -> Clause .

ceq : infer-resolution-pos(CS /\ (p(P, TL1) \/ C1), ~ p(P, TL2) \/ C2) =
  subst(C1 \/ C2, unify(TL1, TL2)) if unify(TL1, TL2) /= no-unifier .
ceq : infer-resolution-neg(CS /\ (~ p(P, TL1) \/ C1), p(P, TL2) \/ C2) =
  subst(C1 \/ C2, unify(TL1, TL2)) if unify(TL1, TL2) /= no-unifier .
ceq : infer-factoring(p(P, TL1) \/ p(P, TL2) \/ C) =
  subst(p(P, TL2) \/ C, unify(TL1, TL2)) if unify(TL1, TL2) /= no-unifier .

ceq : inference-step(CS1, C1, CS2) =
  inference-step(CS1, C1, (C2 /\ CS2))
  if infer-resolution-pos(CS1, C1) = C2 /\ not C2 in CS2 .
ceq : inference-step(CS1, C1, CS2) =
  inference-step(CS1, C1, (C2 /\ CS2))
  if infer-resolution-neg(CS1, C1) = C2 /\ not C2 in CS2 .
ceq : inference-step(CS1, C1, CS2) =
  inference-step(CS1, C1, (C2 /\ CS2))
  if infer-factoring(C1) = C2 /\ not C2 in CS2 .
eq : inference-step(CS1, C1, CS2) = CS2 [owise] .

```

This implementation uses three intermediate functions, two of them apply the resolution rule when the given clause has a positive/negative literal, and one of them that apply the

Statement: “I understand ...”	Strongly Agree	Agree	No Opinion	Disagree	Strongly Disagree	Total
clausification.	3	6	1	2	0	12
substitution.	1	4	3	1	2	11
Skolemisation.	2	6	1	2	1	12
unification.	1	3	2	4	2	12
saturation.	0	2	3	3	4	12
DPLL.	2	6	1	3	0	12
tableaux.	1	5	2	4	0	12

Table 1: Replies to the Likert scale used for student self-assessment, *before* the exercise. For each notion n , the students were asked whether they agreed with the statement “I understand n ”

factoring rule. The main inference-step function applies these three functions until they cannot add more clauses.

Below is a possible solution for the saturation function:

```

op saturation : CNF CNF -> Nat .

eq : saturation(bot /\ NEW, OLD) = 1 .
eq : saturation(top, OLD) = 0 .
eq : saturation(C /\ NEW, OLD) = saturation(inference-step(OLD, C), C /\ OLD) .

```

3 Results

At the beginning of the first session, the students were asked to assess their understanding level of the different notions taught in the course, using a Likert scale with five-level items. Each item featured a statement “I understand n ”, with n a course notion among “clausification”, “substitution”, “Skolemisation”, “unification”, “saturation”, “DPLL” and “tableaux”. The results of that poll are given in Table 1. From these results, we can gather that the students were generally fairly confident that they had understood the material covered in class. For example, 66% of the respondents agreed or strongly agreed with the statement “I understand Skolemisation”, and 75% agreed or strongly agreed with the statement “I understand clausification”. Some topics appeared unclear to many students, for example saturation, with only 2 students (16%) agreeing that they had understood the notion.

One major difficulty encountered by the students was their unfamiliarity with recursive functions. Maude, being based on rewriting logic, shares similarities with functional programming. It is natural to encode algorithm used in the lesson in a recursive way even when the textbook doesn’t specifically says so. For example a lot of time was needed for the the students to find a suitable termination condition for the clausify function.

We had anticipated that Maude programming style would require some effort to learn, but we were hoping that the students' previous experiences in functional programming (mostly with OCaml and Coq) would help. Unfortunately, while the students had previously taken functional programming courses, most had little practice with that style of programming. Many students needed some time to adjust to the equational programming style, let alone make use of Maude's more unique features such as AC unification. Once the difficult first contact was made, students managed to implement the algorithms of the exercises by taking advantage of Maude's capabilities in pattern matching and sub-typing. This was facilitated by the fact that the first functions they needed to implement were the ones that they were the most comfortable with, as seen in Table 1. The Maude environment proved very reliable and simple to use, and no student encountered any technical issue during the two sessions. By the end of the last session, most students had managed to implement the clausification, substitution, and Skolemisation algorithms.

At the end of the last lesson, we provided the students with another evaluation form designed to assess their opinion on these exercises and the tool they had to use. We had initially planned to ask self-assessment questions similar to those asked before the exercise, in order to measure the difference. However since the progress was slower than anticipated, few of the notions listed in the self-assessment were actually covered in the exercises that the students had managed, and we did not include the assessment questions.

From the results in Table 2, we can see that most students found the exercise to be a helpful addition to the course: in contrast with their confidence during the initial self-assessment, 66% of the students agreed or strongly agreed that the implementation exercise helped them realize they had misunderstood some of the material, 66% agreed that it helped them better understand the material. Most of them (77%) also found Maude to be a suitable choice, despite only 33% agreeing that it is easy to use, and even 22% strongly disagreeing with that point. We also asked questions about pattern matching modulo AC, an entirely new feature for these students. While 44% of the respondent has no opinion on this feature, those who did generally found it easy to use and suitable for the task.

4 Conclusion

This paper presents a pedagogical experiment in which students used the Maude programming language to implement automated deduction techniques for a software verification course. The choice of using the Maude programming language was motivated by features such as pattern matching modulo AC and polymorphism, that allow students to write algorithms that closely mirror textbook presentations. This alignment reduces the effort and time needed to turn the formal presentation into an implementation, letting students focus on understanding the algorithms themselves rather than dealing with implementation details.

Our experience, though limited, suggests that Maude can be an effective medium to teach automated deduction algorithms, without adding implementation-related complexity. Students were generally able to engage with the material through practical exercises, and the features of Maude allowed them to concentrate on correctness over implementation mechanics. Their feedback was generally positive indicating that this was a valuable experience for them.

However, several limitations affected the outcomes. Students were unfamiliar with Maude, and the time frame of two sessions was short. As a result, they spent a significant portion of this time learning the language itself rather than engaging with the course content, and only a fraction of the offered material was effectively carried out by the students. Furthermore, the assessment was hindered by the small number of students, and the assessment strategy itself

Statement	Strongly Agree	Agree	No Opinion	Disagree	Strongly Disagree	Total
“This exercise helped me realize I had misunderstood some of the course material.”	3	3	2	1	0	9
“This exercise helped me better understand the course material.”	0	6	2	0	1	9
“Maude is easy to use.”	0	3	3	1	2	9
“Maude is a suitable tool for AR.”	0	7	1	1	0	9
“Pattern matching modulo AC is easy to use.”	1	4	4	0	0	9
“Pattern matching modulo AC is a suitable tool for AR.”	0	4	4	0	1	9

Table 2: Replies to the Likert scale used to assess the relevance of the programming exercise and of the Maude programming language.

could be improved.

We plan to renew the experience in a future iteration of the course next year, allocating more time to it. Ideally, each lecture will be accompanied by an implementation exercise, as is currently the case for pen-and-paper exercise sessions. This should provide students with the time needed to become comfortable with the language, and allow them to implement notions as they learn about them, rather than during two long sessions at the end of the semester.

References

- [1] Ramachandra B Abhyankar. Teaching deduction using Athena and related tools. In *Proceedings of the 49th ACM Technical Symposium on Computer Science Education*, pages 1106–1106, 2018.
- [2] Manuel Clavel, Francisco Durán, Steven Eker, Patrick Lincoln, Narciso Martí-Oliet, José Meseguer, and José F Quesada. Maude: Specification and programming in rewriting logic. *Theoretical Computer Science*, 285(2):187–243, 2002.
- [3] Gila Hanna and Xiaoheng Yan. Opening a discussion on teaching proof with automated theorem provers. *For the Learning of Mathematics*, 41(3):42–46, 2021.
- [4] Pedro Quaresma and Vanda Santos. Four geometry problems to introduce automated deduction in secondary schools. *arXiv preprint arXiv:2202.03566*, 2022.
- [5] Alan JA Robinson and Andrei Voronkov. *Handbook of automated reasoning*, volume 1. Elsevier, 2001.
- [6] Jørgen Villadsen. ProofJudge: Automated proof judging tool for learning mathematical logic. In *Exploring Teaching for Active Learning in Engineering Education (ETALEE 2015)*, pages 141–148. IUPN-Ingeniør Uddannelsernes Pædagogiske Netværk, 2015.